ONNX·TensorRT Day 4

TensorRT 첫걸음 — ONNX 를 엔진으로 바꾸기

HYUNDAI AI Insight Campus · 온디바이스 AI · ONNX·TensorRT 4/5

2026. 11. 9 (월) 09:00–18:00 · 8교시 · Google Colab (T4 GPU 필수)

polygraphy | Builder | OnnxParser | Runtime | FP16 | Inspector

강사 : 김정인 · 김구수

오늘의 학습 목표

같은 일을 두 번 — 명령 한 줄 (Polygraphy) 로 쉽게 , 다음엔 Python 코드로 한 단계씩

1 **엔진이란**

ONNX · ORT · TensorRT 엔진의 차이 , 엔진을 장치마다 다시 빌드하는 이유

2 **명령 한 줄**

polygraphy convert 로 엔진 , polygraphy run 으로 ORT 와 비교 (PASSED · atol)

3 **코드로 빌드**

Logger → Builder → Network → Parser → Config → 저장 → build_engine()

4 **코드로 실행**

Runtime → Context → 이름 · shape → GPU 버퍼 → 실행 → trt_runner()

5 **FP16**

표현 범위 · 정밀도를 이해하고 , 속도와 정확도를 함께 비교한다

6 **왜 빨라졌나**

엔진 인스펙터로 합쳐진 레이어를 찾고 , 빨라진 이유 세 가지를 말한다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 매 교시 보기 → 따라하기 → 바꿔보기

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 3 복습 퀴즈 · TensorRT 란 · 설치 확인	퀴즈 + 실습	D4-1 설계도 · 완성품
2	10:00–10:50	명령 한 줄로 엔진 만들기 — polygraphy convert	강의 + 실습	D4-2 빌드 단계
3	11:00–11:50	명령 한 줄로 비교하기 — polygraphy run	강의 + 실습	D4-3 atol · rtol
4	13:00–13:50	Python 으로 엔진 빌드 → build_engine()	강의 + 실습	D4-4 빌드 여섯 단계
5	14:00–14:50	Python 으로 엔진 실행 → trt_runner()	강의 + 실습	D4-5 엔진 실행
6	15:00–15:50	FP16 으로 더 빠르게 — 속도 · 정확도	강의 + 실습	D4-6 FP32 vs FP16
7	16:00–16:50	왜 빨라졌나 — 엔진 레이어 들여다보기	강의 + 실습	D4-7 레이어 합치기
8	17:00–17:50	미니 프로젝트 — 실행기 4 가지 성능표	개인 실습	D4-8 성능표

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 확인 (노트북) → 점검 (워크시트)

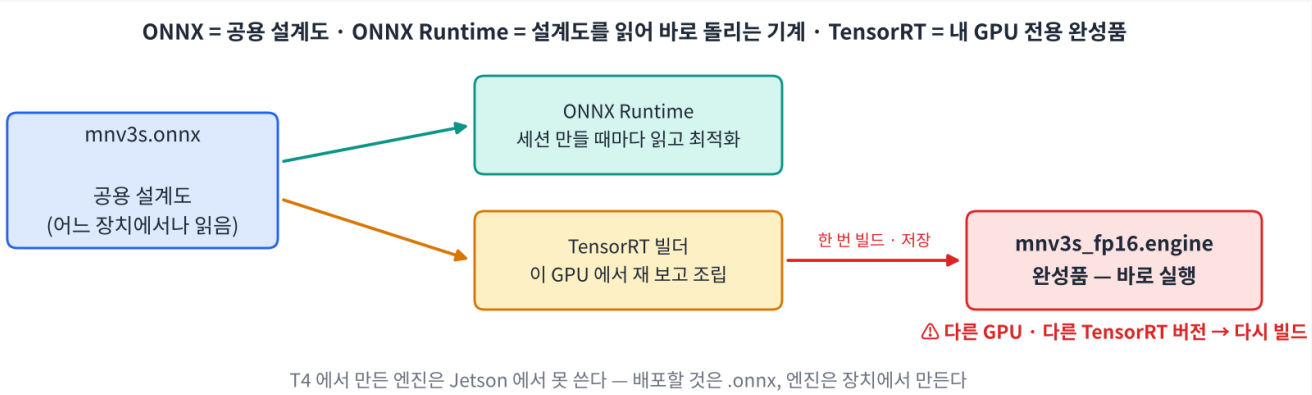
기본 주소 (Day 4)

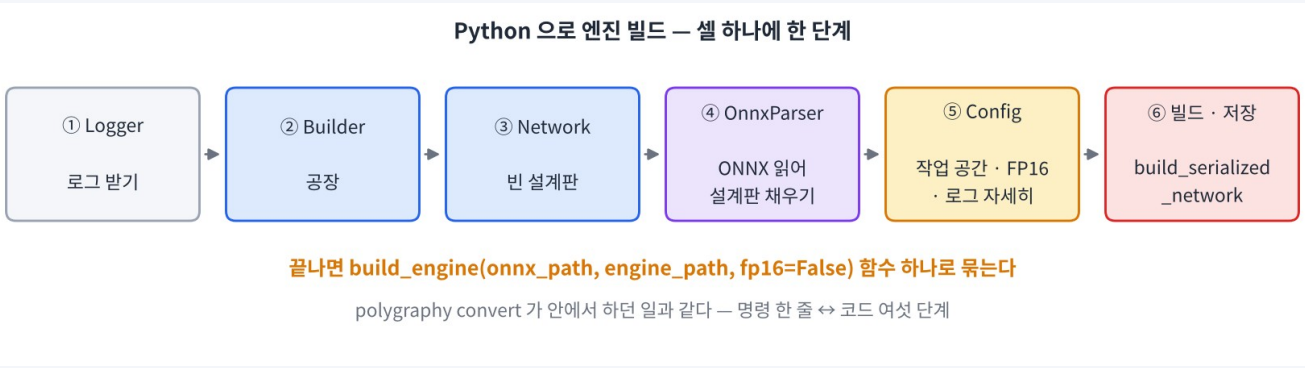
https://imguru-mooc.github.io/AI_CAMPUS/OT-day4/

animation.html	애니메이션 8 개 (D4-1 ~ D4-8) — #d4-1 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기 (교시당 10 문항)
OT_Day4_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab 에서 열기)
OT_Day4.pdf	오늘 슬라이드 PDF
Day 3 복습 퀴즈	1 교시 09:00 — Google Forms 20 문항 (강사가 링크 공유)
런타임 → T4 GPU	오늘은 필수 — GPU 없이는 엔진을 만들 수 없음
ot_baseline.csv	Day 3 파일을 왼쪽 파일 탭에 올려 두기 (8 교시에 이어 붙임)

오늘의 큰 그림 — 설계도 · 기계 · 완성품

엔진은 '이 GPU · 이 TensorRT 버전' 전용 — 배포할 것은 .onnx





오늘의 환경 · 모델

TensorRT 10.16 — 설치 셀

11.x 는 BuilderFlag.FP16 이 없음 → 고정

T4 = sm_75 — 1 교시

compute capability 7.5 · Orin 은 8.7

mnv3s_b1.onnx — 1–8 교시

MobileNetV3 · 배치 1 고정 · 노드 127

mnist_cnn_b1.onnx — 6·8 교시

SimpleConvNet · MNIST 정확도 확인용

ot_baseline.csv — 8 교시

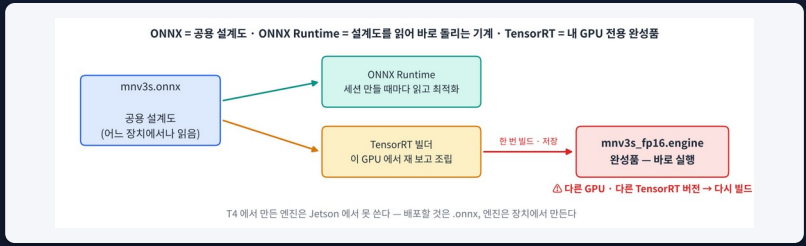
Day 3 성능표에 TensorRT 줄 추가

1 Day 3 복습 · TensorRT 란 · 설치

09:00–09:50

학습 목표

- Day 3 복습 퀴즈 20 문항 (15 분)
- ORT = 매번 최적화, TRT = 한 번 빌드
  - 엔진 = GPU 세대 · TRT 버전 전용
  - 배치 1 고정 ONNX (_b1)



노트북 실습

1. 복습 퀴즈 (Google Forms)
2. 문제 1-1 버전 · GPU 확인
3. 문제 1-2 고정 크기 ONNX

▶ 애니메이션 D4-1 설계도 · 기계 · 완성품

핵심 개념

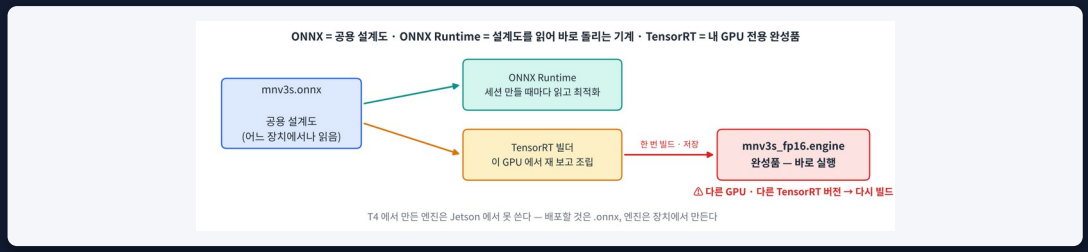
확인

```
trt.__version__              # 10.16.1.11
torch.cuda.get_device_capability(0)
# (7, 5) → sm_75 (T4)
```

고정 크기

```
torch.onnx.export(mnv3s, torch.randn(1, 3, 224, 224),
    "mnv3s_b1.onnx", opset_version=17)
# dynamic_axes 없음 → [1, 3, 224, 224]
```

비유



체크포인트

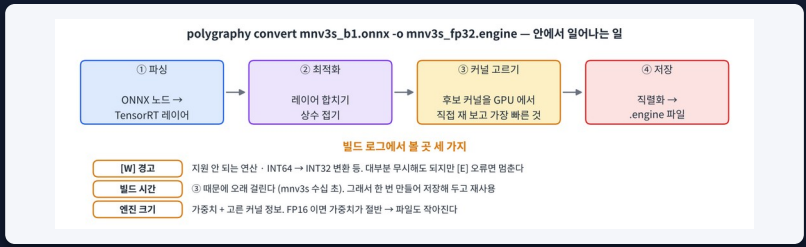
T4 에서 만든 엔진은 Jetson 에서 못 쓴다 — .onnx 를 가져가 장치에서 다시 빌드.

2 명령 한 줄로 엔진 — polygraphy convert

10:00–10:50

학습 목표

- trtexec 대신 Polygraphy (pip 판)
  - 파싱 → 최적화 → 커널 고르기 → 저장
  - 로그 : [E] · [W] · 빌드 시간
- --fp16 한 단어로 FP16 엔진



노트북 실습

1. 문제 2-1 첫 엔진
2. 문제 2-2 로그 세 곳 · 크기
3. 🔁 --fp16

▶ 애니메이션 D4-2 엔진 빌드 단계

핵심 개념

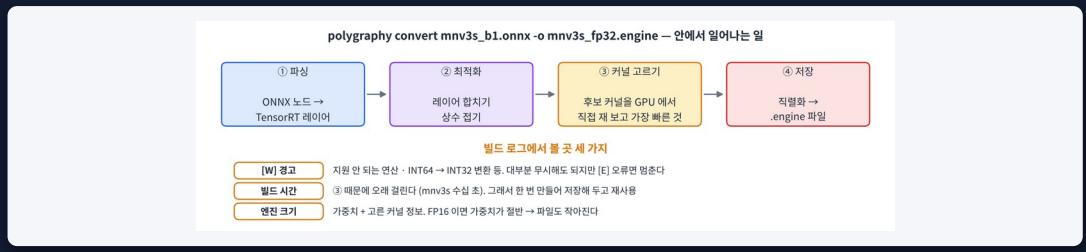
명령

```
polygraphy convert mnv3s_b1.onnx \
    -o mnv3s_fp32_cli.engine
polygraphy convert mnv3s_b1.onnx --fp16 \
    -o mnv3s_fp16_cli.engine
```

로그

```
[W] ...INT64 weights → INT32   # 경고 : 계속
[E] ...                        # 오류 : 실패
... engine building in NN seconds
```

빌드 단계



체크포인트

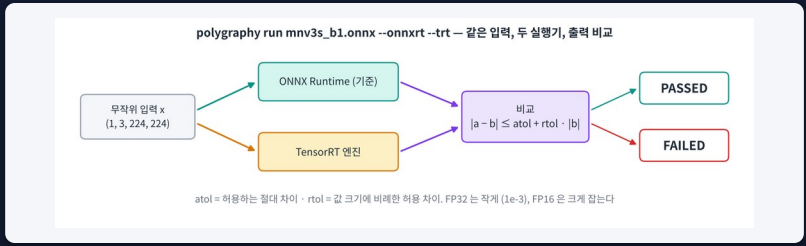
빌드가 오래 걸리는 이유 = 이 GPU 에서 커널 후보를 직접 재 보고 고르기 때문 .

3 명령 한 줄로 비교 — polygraphy run

11:00–11:50

학습 목표

- 같은 무작위 입력 · 두 실행기
- $|a - b| \le atol + rtol \cdot |b|$
- FP32 엄격 (1e-3) → PASSED
- FP16 은 느슨 + 정확도로 판단



노트북 실습

1. 문제 3-1 FP32 비교
2. 문제 3-2 verdict() · FP16 두 기준

▶ 애니메이션 D4-3 비교 · atol · rtol

핵심 개념

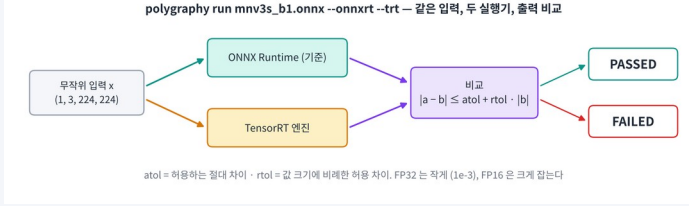
명령

```
polygraphy run mnv3s_b1.onnx --onnxrt --trt \
    --atol 1e-3 --rtol 1e-3
... PASSED | Runtime: ...
```

FP16

```
--trt --fp16 --atol 1e-3   → FAILED 흔함
--trt --fp16 --atol 5e-2   → PASSED ?
# FAILED = ' 기준보다 차이가 크다 '
```

비교



체크포인트

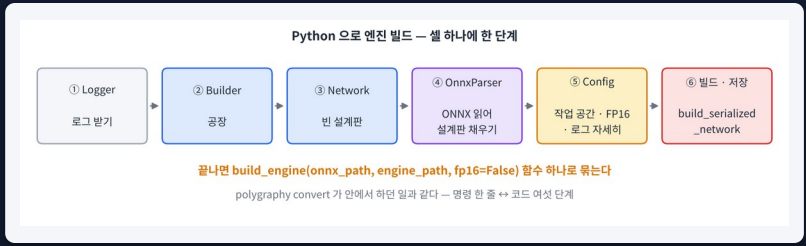
기준을 정하는 것은 사람 — FP16 은 6 교시에 ' 분류 결과가 같은가 ' 로 다시 판단 .

4 Python 으로 엔진 빌드

13:00–13:50

학습 목표

- Logger · Builder · Network
- OnnxParser.parse_from_file → 확인
- Config: WORKSPACE · DETAILED · FP16
- build_serialized_network → 파일



노트북 실습

1. 문제 4-1 파싱
2. 문제 4-2 Config · 빌드 · 저장
3. 문제 4-3 build_engine()

▶ 애니메이션 D4-4 빌드 여섯 단계

핵심 개념

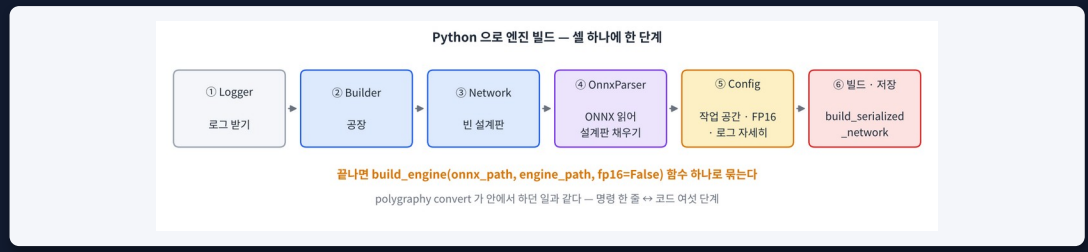
파싱

```
builder = trt.Builder(LOGGER)
network = builder.create_network(0)
parser = trt.OnnxParser(network, LOGGER)
ok = parser.parse_from_file("mnv3s_b1.onnx")
```

빌드

```
config = builder.create_builder_config()
config.set_memory_pool_limit(
    trt.MemoryPoolType.WORKSPACE, 1 << 30)
config.set_flag(trt.BuilderFlag.FP16)  # fp16
blob = builder.build_serialized_network(
    network, config)
```

여섯 단계



체크포인트

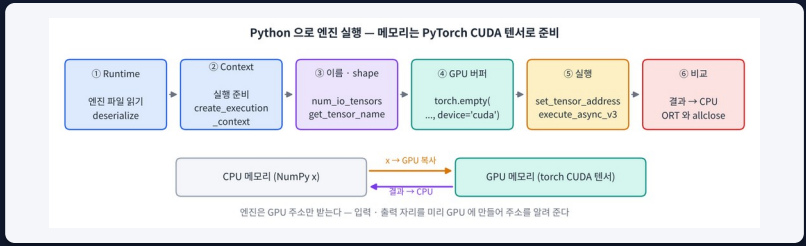
파싱 결과를 꼭 확인 . 빌드마다 Builder · Network · Config 를 새로 만든다 .

5 Python 으로 엔진 실행

14:00–14:50

학습 목표

- Runtime → 엔진 → Context
- get_tensor_name · shape
- GPU 버퍼 = torch CUDA 텐서
- execute_async_v3 → 기다리기



노트북 실습

1. 문제 5-1 입출력 표
2. 문제 5-2 한 번 실행 · 비교
3. 문제 5-3 trt_runner()
4. 문제 5-4 ORT CPU · CUDA · TRT

▶ 애니메이션 D4-5 엔진 실행 — 주소 알려 주기

핵심 개념

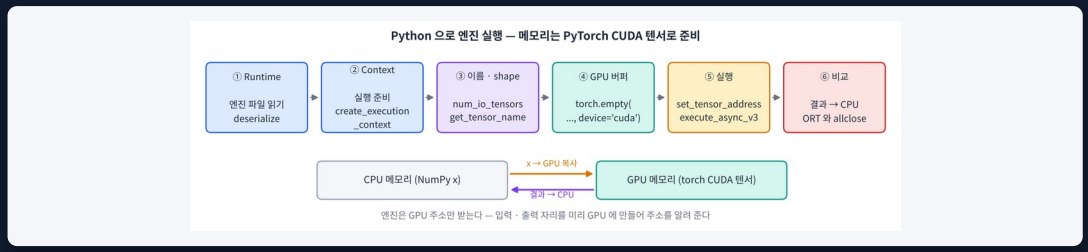
실행

```
d_in = torch.from_numpy(x1).to("cuda")
ctx.set_tensor_address("input", d_in.data_ptr())
ctx.set_tensor_address("logits", d_out.data_ptr())
ctx.execute_async_v3(
    torch.cuda.current_stream().cuda_stream)
torch.cuda.synchronize()
```

비교

```
y = d_out.cpu().numpy()
np.allclose(y, ref, atol=1e-3)  # FP32 → True
```

여섯 단계



체크포인트

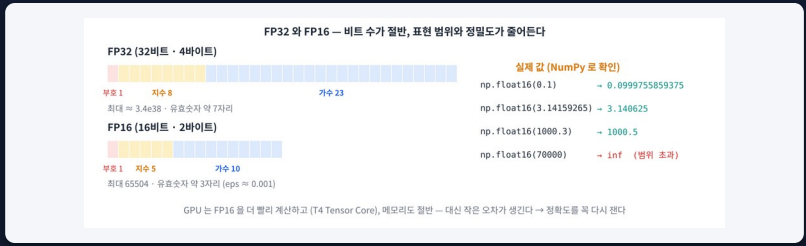
버퍼는 한 번만 만들고 주소도 한 번 . run(x) 는 복사 → 실행 → 기다리기 → 가져오기 .

6 FP16 으로 더 빠르게

15:00–15:50

학습 목표

- FP16 = 2 바이트 · 1/5/10 비트
  - 최대 65504 · eps ≈ 0.001
- fp16=True — Config 한 줄
  - 속도 + 정확도 함께



노트북 실습

1. 문제 6-1 np.float16 만져 보기
2. 문제 6-2 FP16 엔진 · 크기 · 속도
3. 문제 6-3 MNIST 정확도

▶ 애니메이션 D4-6 FP32 vs FP16

핵심 개념

숫자

```
np.float16(0.1)     # 0.0999755859375
np.float16(1000.3)  # 1000.5
np.float16(70000)   # inf
```

정확도

```
acc = {"ORT CPU": ..., "TRT FP32": ...,
       "TRT FP16": ...}   # MNIST 100 장
#  최대 차이 ↑ 여도 분류 결과는 거의 그대로
```

비트



체크포인트

FP16 플래그 = ' 허락 '. 엔진 입출력은 FP32 그대로 — 안쪽 계산만 FP16.

7 왜 빨라졌나 — 엔진 들여다보기

16:00–16:50

학습 목표

- create_engine_inspector()
- JSON → Layers · LayerType
  - 이름의 '+'= 합쳐진 레이어
  - 합치기 · 커널 선택 · FP16



노트북 실습

1. 문제 7-1 레이어 목록
2. 문제 7-2 합쳐진 레이어 세기
3. 문제 7-3 레이어 종류 · CLI

▶ 애니메이션 D4-7 레이어 합치기

핵심 개념

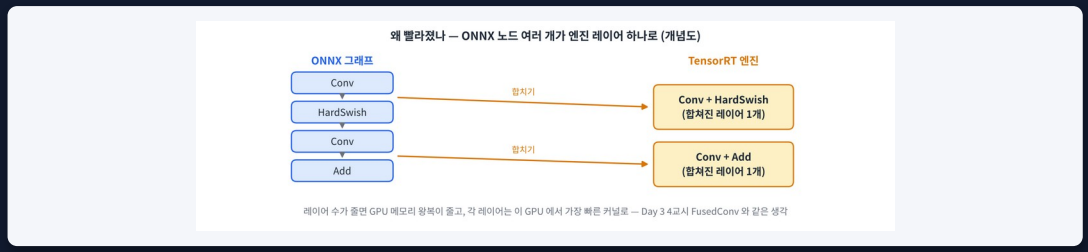
인스펙터

```
insp = engine.create_engine_inspector()
info = json.loads(insp.get_engine_information(
    trt.LayerInformationFormat.JSON))
names = [l["Name"] for l in info["Layers"]]
```

합치기

```
fused = [n for n in names if " + " in n]
# .../Conv + .../HardSwish  → 레이어 1 개
```

개념도



체크포인트

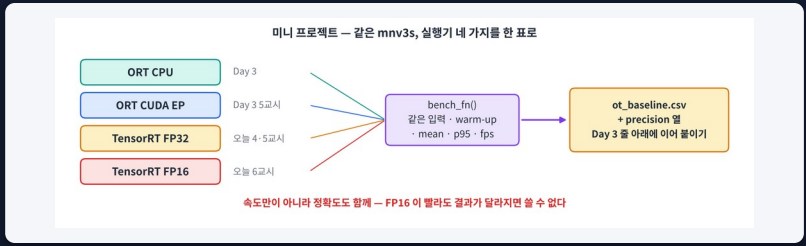
ONNX 노드 127 개 → 엔진 레이어는 훨씬 적다 (실제 개수는 Colab 에서 확인).

8 미니 프로젝트 — 실행기 4 가지 성능표

17:00–17:50

학습 목표

- ORT CPU · ORT CUDA · TRT FP32 · TRT FP16
  - 같은 입력 · 같은 bench_fn
  - 속도 + max_diff + MNIST 정확도
- ot_baseline.csv 에 이어 붙이기



노트북 실습

1. 과제 1 4 행 측정표 (20)
2. 과제 2 정확도 열 (20)
3. 과제 3 MNIST 정확도 (20)
4. 과제 4 speedup · fastest (20)
5. 과제 5 · 6 CSV · 정리 (20)

▶ 애니메이션 D4-8 실행기 4 가지 성능표

핵심 개념

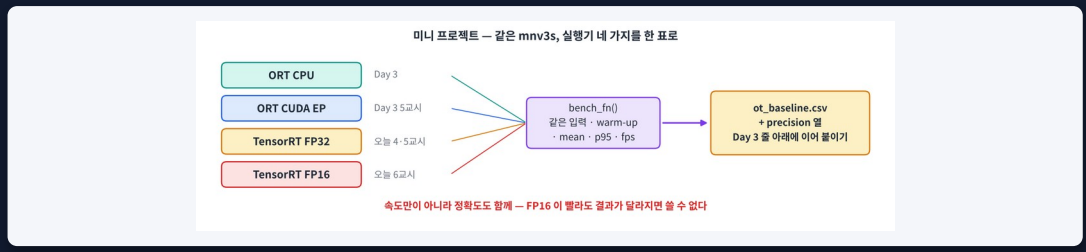
측정

```
for name, f in RUN.items():
    r = bench_fn(lambda: f(x1))
    rows.append({"provider": name,
        "precision": PREC[name], **r})
```

비교

```
cmp["speedup"] = cpu_mean / cmp["mean"]
fastest = cmp.loc[cmp["p95"].idxmin(),
                  "provider"]
```

흐름



체크포인트

채점 셀 자동 90 점 + 과제 6 서술 10 점 . 숫자가 아니라 방법 · 일관성을 채점 .

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

엔진	이 GPU 세대 · 이 TensorRT 버전 전용 완성품 · 배포는 .onnx	D4-1
convert	polygraphy convert -o (+ --fp16) · 로그 [E] · [W] · 빌드 시간	D4-2
run	--onnxrt --trt · PASSED / FAILED · $\|a - b\| \le atol + rtol \cdot \|b\|$	D4-3
빌드	Logger → Builder → Network → Parser → Config → build_serialized_network	D4-4
실행	Runtime → Context → 이름 · shape → CUDA 버퍼 → execute_async_v3 → 기다리기	D4-5
FP16	2 바이트 · 최대 65504 · 유효숫자 3 자리 · 속도와 정확도를 함께	D4-6
빨라진 이유	레이어 합치기 · 이 GPU 에서 고른 커널 · FP16 Tensor Core	D4-7
성능표	실행기 4 가지 · 속도 + 정확도 · ot_baseline.csv	D4-8

Day 5 로 가는 다리 — 탐지 모델 · 침입 감지

오늘의 함수 두 개를 클래스로 묶어 YOLOv8n 에 그대로

Day 4 (오늘)		Day 5 (다음 시간)
`build_engine()` · `trt_runner()`	→	TRTInfer 클래스 (__init__ 로드 · __call__ 실행)
`mnv3s_b1.onnx (1×1000)`	→	yolov8n.onnx → FP16 엔진 (1×84×8400)
`MNIST 정확도로 FP16 확인`	→	FP32 vs FP16 mAP 비교
`bench_fn — 추론만`	→	전처리 · 추론 · 후처리 · 그리기 나눠 재기
`VB Day 5 침입 감지기`	→	sess_yolo → TRTInfer 한 곳만 교체
`ot_baseline.csv`	→	YOLOv8n TensorRT 줄 추가 · 최종본

노트북 · ot_baseline.csv 를 Drive 에 저장해 두세요 — Day 5 에서 이어 씁니다